

Para hacer una traduccion de pseudocodigo a python hay que entender la diferencia que estos tienen.

Diferencia General

El pseudocodigo es una representacion cercana al lenguaje humano de la logica de un algoritmo y su gramatica es bastante flexible.

Mientras que python es un lenguaje con su gramatica mas rigida, orientada a una mayor cercania para ser interpretada por la maquina.

Instrucciones

Inicio/fin del algoritmo

```
Algoritmo Suma
  leer a
  leer b
  c <- a + b
  Escribir c
FinAlgoritmo
```

Python usa el inicio y fin del archivo como inicio y fin del algoritmo por lo que no usa declaraciones de inicio y fin de algoritmo

Asignacion

```
variable <- 2
```

```
variable = 2
```

Lectura

```
leer entrada
```

```
entrada = input(salida)
```

python cuenta con la ventaja de tener una salida en su comando de input por lo que si `salida = "Ingrese Numero: "` entonces es posible que el resultado final al escribir y oprimir enter sea `Ingrese Numero: 12` siendo ahora `entrada = "12"` definido por el input, siempre se recibe una cadena de texto

Salida

```
mostrar salida
```

```
print(salida)
```

Condicional

```
si (condicion = 1)
    mostrar "Es uno"
sino
    mostrar "No es uno"
finsi
```

```
if condicion == 1:
    print("Es uno")
elif condicion == 2:
    print("Es dos")
else:
    print("No es ninguna de las anteriores")
```

Python tiene la ventaja de cubrir mas condiciones.

Casos especificos

Notese que `elif` solo es valido teniendo un `if` mencionado antes

```
condicion = 1
if condicion == 1:
    print("condicion 1")
elif condicion == 1:
    print("condicion 2")
```

la salida seria:

```
condicion 1
```

pero si

```
condicion = 1
if condicion == 1:
    print("condicion 1")
if condicion == 1:
    print("condicion 2")
```

entonces la salida sera

```
condicion 1
condicion 2
```

teniendo en cuenta eso, entonces si

```
condicion = 1
if condicion == 1:
```

```
    print("condicion 1")
if condicion == 2:
    print("condicion 2")
else:
    print("condicion 3")
```

la salida sera:

```
condicion 1
condicion 3
```

Ciclo repetitivo mientras

```
mientras (variable = Verdadero) hacer
    mostrar salida
finmientras
```

```
while variable == True:
    print(salida)
```

Ciclo repetitivo Hasta que

```
Repetir
    Mostrar "ciclo"
Hasta que variable = Verdadero
```

```
while True:
    print("ciclo")
    if variable == True:
        break
```

Ciclo repetitivo para

```
para variable <- 1 Hasta 5 con paso 1 hacer // incluye el 5
  mostrar variable
finpara
```

```
1
2
3
4
5
```

```
for variable in range(1,5,1): # No incluye el 5
    print(variable)
```

```
1
2
3
4
```

Definicion de funciones

```
funcion c <- restar (a,b)
  c = a + b
finfuncion
```

```
def restar(a,b):
    c = a - b
    return c
    print("Este mensaje nunca se mostrara")
```

llamado de funciones

En ambos medios el llamado es igual

```
resultado = restar(1,2)
```

```
resultado = restar(3,1) # -> 2  
# o tambien  
resultado = sumar(b=1, a=3) # -> 2
```

Tipos de datos

Cadena de texto

```
"texto"  
'texto'
```

```
"texto"  
'texto'  
# Usando str() se puede convertir distintos tipos de dato a  
cadena de texto  
str(1)  
str(None)  
str(True)  
str([])
```

Numero Entero

```
2
```

```
2  
int("2") # Solo funciona si la cadena de texto es un numero
```

```
#Para comprobar si la cadena de texto es un numero  
"123".isnumeric() # -> True
```

Dicotomia (verdadero o falso)

Verdadero
Falso

```
True # no funciona si es true  
False # no funciona si es false  
bool(1) # -> True  
bool(0) # -> False  
bool("Texto") # -> True  
bool("") # -> False  
bool([1,2,3]) # -> True  
bool([]) # -> False
```

Arreglos/Listas

dimensionar lista[4] // Primero se debe reservar cuantos elementos maximo va a tener la lista/arreglo

```
lista[1] = 1  
lista[2] = 2  
lista[3] = 3  
lista[4] = "hola" // Error, el arreglo debe tener elementos de un solo tipo
```

```
lista = [None, True, 1, [1,2,"nombre"], "hola"] # se pueden  
mezclar distintos tipos de valores y no es necesario  
reservar memoria para la lista previamente
```